# RecordBuf : UGen : AbstractFunction : Object

*Record or overdub into a Buffer.*

Source: BufIO.sc

**See also: PlayBuf**

## Description

Records input into a Buffer.

If recLevel is 1.0 and preLevel is 0.0 then the new input overwrites the old data. If they are both 1.0 then the new data is added to the existing data. (Any other settings are also valid.)

**NOTE:** The number of channels must be fixed for the SynthDef, it cannot vary depending on which buffer you use.

## Class Methods

```
RecordBuf.ar(inputArray, bufnum: 0, offset: 0.0, recLevel: 1.0, preLevel: 0.0,
       run: 1.0, loop: 1.0, trigger: 1.0, doneAction: 0)

RecordBuf.kr(inputArray, bufnum: 0, offset: 0.0, recLevel: 1.0, preLevel: 0.0,
       run: 1.0, loop: 1.0, trigger: 1.0, doneAction: 0)
```

**Arguments:**

| | |
|---|---|
| **inputArray** | An Array of input channels. |
| **bufnum** | The index of the buffer to use. |
| **offset** | An offset into the buffer in samples. |
| **recLevel** | Value to multiply by input before mixing with existing data. |
| **preLevel** | Value to multiply to existing data in buffer before mixing with input. |
| **run** | If zero, then recording stops, otherwise recording proceeds. |
| **loop** | If zero then don't loop, otherwise do. This is modulatable. |
| **trigger** | a trigger causes a jump to the start of the Buffer. A trigger occurs when a signal changes from negative value to positive value. |
| **doneAction** | an integer representing an action to be executed when the buffer is finished recording. This can be used to free the enclosing synth, etc. See Done for more detail. `doneAction` is only evaluated if loop is 0. |

### Inherited class methods

**7 methods from UGen** ► show

**8 methods from Object** ► show

## Instance Methods

### Inherited instance methods

**97 methods from UGen** ► show

**161 methods from AbstractFunction** ► show

## Examples

```
// Execute the following in order
(
// allocate a Buffer
s = Server.local;
b = Buffer.alloc(s, 44100 * 4.0, 1); // a four second 1 channel Buffer
)

// record for four seconds
(
SynthDef(\help_RecordBuf, { |out = 0, bufnum = 0|
    var formant;
    formant = Formant.ar(XLine.kr(400, 1000, 4), 2000, 800, 0.125);
    RecordBuf.ar(formant, bufnum, doneAction: Done.freeSelf, loop: 0);
}).play(s, [\out, 0, \bufnum, b]);
)

// play it back
(
SynthDef(\help_RecordBuf_playback, { |out = 0, bufnum = 0|
    var playbuf;
    playbuf = PlayBuf.ar(1, bufnum);
    FreeSelfWhenDone.kr(playbuf); // frees the synth when the PlayBuf is finished
    Out.ar(out, playbuf);
}).play(s, [\out, 0, \bufnum, b]);
)

// overdub
(
SynthDef(\help_RecordBuf_overdub, { |out = 0, bufnum = 0|
    var formant;
    formant = Formant.ar(XLine.kr(200, 1000, 4), 2000, 800, 0.125);
    // mixes equally with existing data
    RecordBuf.ar(formant, bufnum, 0, 0.3, 0.5, doneAction: Done.freeSelf, loop:
0);
}).play(s, [\out, 0, \bufnum, b]);
)

// play back the overdubbed version
Synth.new(\help_RecordBuf_playback, [\out, 0, \bufnum, b], s);

// write the contents of the buffer to a file (see Buffer for more options)
(
b.write(sampleFormat: 'int16');
thisProcess.platform.recordingsDir +/+ "SC_" ++ Date.localtime.stamp ++ ".aiff";
// generated path
)

b.close; b.free; // cleanup
```

helpfile source: /usr/local/share/SuperCollider/HelpSource/Classes/RecordBuf.schelp
link::Classes/RecordBuf::